Team A2 | Data Cleaning Handoff

`Data/processed/recipes_clean.csv` is in the repo. It filters nothing and fills in nothing, so each of you filters it for your own question in the second cell of your notebook.

How the cleaned file was built

The rule behind every decision: **clean, don't filter. Flag, don't fill.** Two questions need different subsets of the same rows, so the file corrects what is genuinely wrong, marks what is questionable with a true/false column, and leaves every row standing.

What happened to the two raw files:

1. **They were stacked, not joined.** 1,163 recipes in `cuisines.csv` appear nowhere in `all_recipes.csv`, and only 1,055 URLs overlap. A join would have discarded the 1,163. The result is 15,589 unique recipes, keyed on `url`.
2. **`country` was mapped on** where it exists, with `has_cuisine` marking the 2,218 rows that have it.
3. **Impossible nutrition values became blanks.** Anything over 2,000 calories, 200g fat, 300g carbs or 100g protein *per serving* is a batch total mislabelled, not a recipe. 16 recipes had protein over 100g, topping out at 939g. Those cells are now NaN and `nutrition_flag` is True.
4. **Impossible cooking times became blanks.** 360 recipes claimed zero total time, and 11 had a total shorter than prep plus cook. `time_flag` marks them.
5. **Missingness was marked, not removed:** `rating_missing` (972 rows), `nutrition_missing`.
6. **Ingredient text was lowercased and stripped, and nothing else.** No splitting, no counting, no columns.

What the file deliberately does NOT contain, and why it matters to you: no imputed values, no scaling, no PCA, no ingredient dummy columns. Every one of those is *fitted* to the data, meaning it learns something from the rows it sees. Run once over the whole file and that learning crosses the train/test boundary before anyone splits anything. §5.3 names fitting a scaler before splitting as the example of what not to do, and item 3 of the rubric is 20 points, the heaviest on it.

So those steps happen inside your notebook, on training folds only. That is extra work for you and it is not optional.

Setup: the first five cells

Same shape as every lab. Libraries, then load, then look at what you loaded. Keep a plain-English comment above each cell the way the labs do; §6.3 checks your notebook for exactly that.

Cell 1, libraries. Start from the lab1 and lab2 block and add only what your own method needs:

```
# Load the libraries used in this notebook.
%matplotlib inline
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Cell 2, load the data. One line, exactly like the labs load UniversalBank and Universities:

```
# Load the cleaned recipe data from the team repository and look at the first few
rows.
recipes_df =
pd.read_csv('https://raw.githubusercontent.com/jalmendral/BA305-A1-TeamA2/main/Data/p
rocessed/recipes_clean.csv')
recipes_df.head()
```

Note the capital **D** in `Data`. If the repository's default branch is `master` rather than `main`, change that word. This only works while the repo is public; if it is private the URL returns a 404 and we switch to cloning.

Cell 3, check what you got. The same two calls every lab makes:

```
# Check the size of the table and the type of each column.
print(recipes_df.shape)
recipes_df.info()
```

Cell 4, your name and your slice. The name comment is required by §6.3 and must match your row in the contribution table.

```
# --- Kenny: regression tree and random forest, baseline, metric choice ---
# Keep only recipes that actually have a rating to predict.
pred_df = recipes_df[~recipes_df.rating_missing].copy()
print(pred_df.shape)        # 14,559 rows
```

```
# --- Justin: ridge and lasso regression, neural network ---
pred_df = recipes_df[~recipes_df.rating_missing].copy()
print(pred_df.shape)        # 14,559 rows
```

```
# --- Jillian: PCA scaled and unscaled, scree plot, component interpretation ---
# Keep only recipes that carry a cuisine label.
desc_df = recipes_df[recipes_df.has_cuisine].copy()
print(desc_df.shape)        # 2,218 rows, 49 cuisines
```

```
# --- Grant: structured visual analysis of nutrition by cuisine ---
desc_df = recipes_df[recipes_df.has_cuisine].copy()
print(desc_df.shape)        # 2,218 rows, 49 cuisines
```

Cell 5, define X and y, the way lab3 and lab6 do it:

```
# Define X (inputs/predictors) and y (the outcome we want to predict).
X = pred_df['ingredients']
y = pred_df['avg_rating']
```

GitHub caches raw files for a few minutes. If the cleaned file was just updated and you still see the old one, wait a moment and re-run cell 2.

What is in the file

Column	Type	Use it as
`url`	text	unique key, never a feature. An identifier is not information about the case (§5.3)
`name`	text	reference only, 25 duplicate names but no duplicate URLs
`author`	text	not a feature as-is. 8,297 distinct values over 15,589 rows
`date_published`, `year`	date, int	missingness checks, possible feature with an argument
`country`	nominal	the descriptive target variable. 49 cuisines, 2,218 rows
`has_cuisine`	bool	filter for the descriptive track
`ingredients`	text	predictive input, vectorized in your notebook
`calories`, `fat`, `carbs`, `protein`	numeric	descriptive inputs. Contains NaN
`avg_rating`	numeric	the predictive target. 972 NaN
`total_ratings`, `reviews`	numeric	**LEAKAGE. Never a feature.** Both recorded after a rating exists
`prep_time`, `cook_time`, `total_time`	numeric	optional features. NaN where `time_flag`
`servings`	numeric	context for per-serving nutrition. 21 NaN
`rating_missing`, `nutrition_missing`, `nutrition_flag`, `time_flag`	bool	filtering and error analysis, never features

On the leakage columns: a model using `total_ratings` would predict a rating from its own popularity and look excellent for no reason you could explain. §5.3 requires the report to name at least one feature dropped for leakage. These are the two, and the reason is chronological: they did not exist at the moment the prediction would have to be made.

Predictive team: Kenny and Justin

Question: can the rating of a recipe be predicted from its ingredients?

Which methods count. §5.2 lists the eligible predictive methods as k-NN, Naïve Bayes, logistic regression, decision trees, regression trees and neural networks. So the two required methods are **Kenny's trees** and **Justin's neural network**. Ridge and lasso are not on that list and are taught in no

lecture or lab, which makes them the **optional third method** §5.2 allows. That is a fine place for them, but it means the neural network is load-bearing, not a stretch goal, and nothing in the course backs it.

One useful consequence: if the classification fork triggers, ridge and lasso become `LogisticRegression(penalty="l1")` or `"l2"`, which **is** on the §5.2 list and is taught in Lecture 9 and lab7. Justin's work would stop being an off-list extra.

Expect a weak answer. A rough forest on binary ingredient indicators explained roughly 4% of rating variation. Ratings pile up between 4.3 and 5.0 with a standard deviation of 0.41 stars. Verify this yourselves in the first hour. It changes the plan, it does not end it: item 4 gives **full marks** for honestly establishing that a question cannot be answered well from this data, with evidence and a diagnosis. Quietly swapping in an easier question scores zero.

Agree these three before either of you fits anything, or the models are not comparable and §5.4 requires them to be:

- ☐ Same split: `train_test_split(X, y, test_size=0.2, random_state=42)`
- ☐ Same metric: RMSE in stars, reported alongside R^2. Write the sentence saying why it is the right one.
- ☐ Same baseline: `DummyRegressor()` predicting the mean, run **first**, before any model. That is the number all three models must beat.

The ingredient matrix. The text becomes one 0/1 column per ingredient word, which is an ordinary feature table. It is built **inside a Pipeline** so the vocabulary is learned per fold rather than once across everything:

```
# Turn the ingredient text into one yes/no column per ingredient word.
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.dummy import DummyRegressor

STOP = ["cup","cups","teaspoon","teaspoons","tablespoon","tablespoons",
        "ounce","ounces","pound","pounds","package","can","large","medium",
        "chopped","minced","sliced","ground","fresh","taste","needed",
        "and","all","purpose","optional","divided","softened"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

vec = CountVectorizer(binary=True, min_df=20,
                      token_pattern=r"[a-zA-Z]{3,}", stop_words=STOP)
```

Vocabulary size on this data: `min_df=10` gives 1,391 features, `min_df=20` gives 1,039, `min_df=50` gives 681. Start at 20 and tune it as a hyperparameter.

Kenny: regression tree, then random forest. Lab6 already runs `DecisionTreeRegressor` with `mean_squared_error` and `r2_score`, so the pattern is there to copy; `RandomForestRegressor` is one import away from lab6's classifier. Your contrast is a single interpretable tree against the ensemble, and §5.4 asks which you would deploy and why. Tune `max_depth` and `min_samples_leaf`, 5 to 10 values

each, by cross-validation on training data only. Widen the range if the best value sits at the edge. Your single tree is the one that can be drawn on a slide, which is why it exists alongside the forest.

Justin: ridge and lasso. Both are linear regression with a penalty that shrinks coefficients, and lasso shrinks some to exactly zero, which is automatic feature selection over roughly a thousand ingredient columns. The payoff for §5.5 is that coefficients are **signed**: forest importance says vanilla matters, lasso says whether it raises or lowers the rating. Tune `alpha` across a wide log range plus the vectorizer's `min_df`, which covers the two-hyperparameter requirement. Binary dummies are already on a common 0-to-1 scale, so no scaler is needed for them alone; add one to the pipeline the moment you include `calories` or `prep_time`.

```
# Pull out the ingredients with the largest positive and negative effects.
coef = pd.Series(best.named_steps["model"].coef_,
                 index=best.named_steps["vec"].get_feature_names_out())
print(coef.sort_values().head(15))    # lowers the rating
print(coef.sort_values().tail(15))    # raises the rating
```

Justin: neural network. `MLPRegressor` on the same matrix. No lab covers this, so budget real time and be ready to explain in one plain sentence what the network is doing. Lecture 10 slide 13 is the reference: all features numeric, all features scaled, overfitting is the danger. Tune hidden layer size and `alpha`, and use `early_stopping=True`.

All three models also owe:

- ☐ A fit diagnosis with evidence, not assertion: training score next to test score, plus a curve across the tuning range
- ☐ Error analysis in the three steps of §5.4. Check recipes you can reason about, compare what the model gets wrong against what it gets right, say what the pattern suggests. Likely suspects: recipes with very few ratings, and recipes with unusual ingredient counts.
- ☐ A grouped-ingredient run as a contrast. Map tokens to a dozen categories (dairy, red meat, baking, spices) and compare against the thousand-dummy version. Given how thin the signal is, that comparison may be the most interesting thing in this section.
- ☐ The classification fork. If nothing beats the baseline meaningfully by Nov 6, bin `avg_rating` into low/medium/high and model that. Report both.

Descriptive team: Jillian and Grant

Question: do nutritional profiles separate national cuisines, or do many cuisines collapse into a few nutritional profiles?

You work on `desc_df`: 2,218 recipes with a `country` label across 49 cuisines. Ratings are irrelevant here, which is exactly why the cleaned file kept the 972 unrated recipes. Their nutrition is fine.

Your two methods are now the two §5.2 asks for - structured visual analysis and PCA - with nothing off-list and nothing untaught. That is worth protecting. Clustering is out, so the word **collapse** in your question has to be demonstrated some other way: overlapping distributions in Grant's boxplots, and cuisines sitting on top of each other in Jillian's PC1-PC2 scatter. Decide together, before you build anything, what "collapse" will look like as evidence. If you cannot say in advance, the question is not yet answerable.

Two things to settle in your first session:

- [ ] The small-cuisine problem. Belgian has 6 recipes, Swiss 10, Colombian 11, Bangladeshi 12. A boxplot of 6 points is not a distribution. Decide whether cuisines under roughly 20 recipes are pooled, excluded or kept, write down the argument, and apply the same rule in both notebooks.
- [ ] Which numeric features are in play. Calories, fat, carbs and protein are the obvious four. Whether prep time and servings join them is a decision, not a default, and it changes what PC1 means.

Scaling is not in the cleaned file, deliberately. Calories run in the hundreds and protein in the tens. PCA works on variance, so unscaled, calories would dominate everything. Lecture 3 makes rescaling a central point, and lab2 shows `StandardScaler` doing it. Fit it in your own notebook.

Jillian, PCA. Lab2 is the template end to end.

- [ ] Numeric features only. §5.4 warns that one-hot encoding a category and feeding it to PCA silently asserts every pair of categories sits the same distance apart. `country` is the thing you are explaining, not an input.
- [ ] Run it scaled and unscaled and argue for one. §5.2 accepts the same method under two genuinely different choices as a valid method pair, which is how PCA alone carries its share.
- [ ] Justify the component count with a scree plot and cumulative explained variance, using the eigenvalue above 1 and percentage-of-variance criteria from Lecture 3 slide 24. "Two because it plots nicely" is named in the directives as a non-argument.
- [ ] Name what each component means in the language of food. A component loading on calories and fat together is a richness axis. Say that, not "PC1".
- [ ] The headline figure is PC1 against PC2 coloured by cuisine. Build it early. It tells Grant which cuisines are worth putting side by side in a chart.

Grant, structured visual analysis. This is a required method under §5.2, not a warm-up. Lab1 is your template, and the Session 3 laptop case is the argument you are expected to reproduce.

- [ ] A deliberate set of figures, not one scatter plot: nutrition distributions by cuisine as **boxplots** so the spread shows, plus a third variable brought in by colour or one panel per group.
- [ ] Apply the laptop-case test to every figure you rely on. A bar of group means can say the cuisines separate while a boxplot of the same data shows the ranges swamp the difference. Same data, opposite conclusion. If a different but equally reasonable chart would support a different reading, show both and say which is honest.
- [ ] Use a log scale where a skewed nutrition variable would let one extreme recipe draw the whole picture. §5.4 treats that as a claim you are making, not a formatting choice.
- [ ] Check for a third variable. Does a cuisine look high-calorie because of its food, or because its recipes list more servings? Bring `servings` into the chart and see whether the pattern survives.
- [ ] Say so if the structure is weak. §5.4 says a thin result honestly reported scores better than a picture squinted at until it agrees with you, and "most of these cuisines are nutritionally indistinguishable" is a real finding.

Rules for everyone

Your notebook is yours alone. Colab's *Save a copy in GitHub* replaces the whole file, with no merge and no warning. Open a notebook with someone else's name on it and save, and their work is gone. Always open from GitHub, never from an old Colab tab, and say in the group chat when you have saved.

Commit messages are graded evidence. §6.3 checks the commit history against the contribution table in the report. Replace Colab's default "Created using Colab" every time. `tune max_depth 2-20, best at 8` costs the same keystrokes as `update` and is worth something.

Nothing fitted touches data outside your training folds. Imputers, scalers, vectorizers, PCA. Inside `Pipeline`, scored by cross-validation. This is item 3, 20 points, the heaviest on the rubric.

Keep what failed. §5.5 says failed attempts count as much as successful ones: the method that underperformed, the feature that did not help, the fix that made things worse. Keep a running note in your notebook as you go. Reconstructing it on Nov 22 is how teams end up guessing.

Label AI-assisted code as you write it. §9 requires every such block marked in the notebook and disclosed in an appendix with tool, version, how it was used, and links or screenshots. Generating prose for the report or the slides is prohibited outright.

Caption every figure with its conclusion. Axis labels, a legend where applicable, and one line saying what the reader should take from it. §5.5 calls a figure without a takeaway decoration.

Restart and Run All before you save. Final notebooks must run top to bottom from a fresh runtime with every output visible. If the code does not run, item 3 is capped at *Minimal* on top of whatever item 7 loses.

Due to the team by Wed Nov 4, so there is something real at Workshop 2 on Nov 6: Kenny and Justin, a baseline plus one untuned model each on the same split. Jillian, the PC1-PC2 scatter coloured by cuisine. Grant, nutrition boxplots by cuisine.